

CAPÍTULO 1

Conjuntos e linguagens

Neste primeiro capítulo, revisamos algumas propriedades básicas dos conjuntos e suas operações e introduzimos o conceito de linguagem formal.

1. Exercícios

1. Sejam A e B conjuntos, prove que:

- (a) $\emptyset \cup A = A$;
- (b) $\emptyset \cap A = \emptyset$;
- (c) se $A \subset B$ então $A \cap B = A$;
- (d) se $A \subset B$ então $A \cup B = B$;
- (e) $A \cap A = A = A \cup A$;
- (f) $A \cup B = B \cup A$ e $A \cap B = B \cap A$;
- (g) $A \cup (B \cap C) = (A \cup B) \cap C$;
- (h) $A \cap (B \cup C) = (A \cap B) \cup C$;
- (i) $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$;
- (j) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$;
- (k) $A \setminus B = A \cap \overline{B}$ onde $\overline{B}$ é o complemento de B no conjunto *universo*, isto é o conjunto que contém todos os conjuntos com que estamos trabalhando;
- (l) $\overline{(A \cap B)} = \overline{A} \cup \overline{B}$;
- (m) $\overline{(A \cup B)} = \overline{A} \cap \overline{B}$;
- (n) $\overline{\overline{A}} = A$;
- (o) $A \setminus B = A \setminus (B \cap A)$;
- (p) $B \subset \overline{A}$ se, e somente se, $A \cap B = \emptyset$;
- (q) $(A \setminus B) \setminus C = (A \setminus C) \setminus (B \setminus C) = A \setminus (B \cup C)$;
- (r) $A \cap \overline{B} = \emptyset$ e $\overline{A} \cap B = \emptyset$ se e somente se $A = B$.

2. Considere as afirmações abaixo: prove as verdadeiras e dê um contra-exemplo para as falsas.

- (a) se $A \cup B = A \cup C$ então $B = C$;

- (b) se $A \cap B = A \cap C$ então $B = C$.
3. Sejam A , B e C conjuntos, prove que:
- $A \times (B \cap C) = (A \times B) \cap (A \times C)$;
 - $A \times (B \cup C) = (A \times B) \cup (A \times C)$;
 - $A \times (B \setminus C) = (A \times B) \setminus (A \times C)$;
4. Explique a diferença entre $\emptyset$ e $\{\epsilon\}$. Calcule $L \cdot \emptyset$, onde L é uma linguagem qualquer.
5. Seja w uma palavra em um alfabeto Σ . Definimos o *reflexo* de w recursivamente da seguinte maneira: $\epsilon^R = \epsilon$ e se $w = ax$ então $w^R = x^R a$ onde $a \in \Sigma$.
- Determine $(turing)^R$ e $(anilina)^R$.
 - Se x e y são palavras no alfabeto Σ , determine $(xy)^R$ em função de x^R e y^R .
 - Determine $(x^R)^R$.
6. Sejam L_1 e L_2 linguagens no alfabeto Σ . Determine as seguintes linguagens em função de L_1^R e L_2^R :
- $(L_1 \cdot L_2)^R$;
 - $(L_1 \cup L_2)^R$;
 - $(L_1 \cap L_2)^R$;
 - $\overline{L_1}^R$;
 - $(L_1^*)^R$.
7. Mostre, por indução em n , que se $L_0, \dots, L_n$ são linguagens no alfabeto Σ então

$$L_0 \cdot (L_1 \cup \dots \cup L_n) = (L_0 \cdot L_1) \cup \dots \cup (L_0 \cdot L_n).$$

8. Sejam Σ_1 e Σ_2 dois alfabetos e seja $\phi : \Sigma_1 \rightarrow \Sigma_2^*$ uma aplicação. Estenda ϕ a Σ_1^* de acordo com a seguinte definição recursiva:
- $\phi(\epsilon) = \epsilon$;
 - $\phi(x\sigma) = \phi(a)\phi(\sigma)$, onde $a \in \Sigma_1$.
- Se L é uma linguagem no alfabeto Σ_1 defina

$$\phi(L) = \{\phi(w) : w \in \Sigma_1^*\}.$$

Mostre que se L e L' são linguagens no alfabeto Σ_1 então:

- $\phi(L \cup L') = \phi(L) \cup \phi(L')$;
- $\phi(L \cap L') = \phi(L) \cap \phi(L')$;
- $\phi(L \cdot L') = \phi(L) \cdot \phi(L')$;

Autômatos finitos determinísticos

Neste capítulo introduzimos a noção de autômato finito determinístico através de um exemplo concreto e estudamos alguns outros exemplos que ocorrerão freqüentemente ao longo do livro.

1. Exercícios

1. Seja $\mathcal{A}$ um autômato finito determinístico. Quando é que $\epsilon \in L(\mathcal{A})$?
2. Desenhe o grafo de estados e determine a linguagem aceita por cada um dos seguintes autômatos finitos. Em cada caso o estado inicial é q_1 e o alfabeto é $\{a, b, c\}$

(a) $F_1 = \{q_5\}$ e a função de transição é dada por:

δ_1	a	b	c
q_1	q_2	q_3	q_4
q_2	q_2	q_4	q_5
q_3	q_4	q_3	q_5
q_4	q_4	q_4	q_5
q_5	q_4	q_4	q_5

(b) $F_2 = \{q_4\}$ e $\delta_2 = \delta_1$.

(c) $F_3 = \{q_2\}$ e a função de transição é dada por:

δ_3	a	b	c
q_1	q_2	q_2	q_1
q_2	q_3	q_2	q_1
q_3	q_1	q_3	q_2

3. Considere o autômato finito determinístico no alfabeto $\{a, b\}$, com estados $\{q_0, q_1\}$, estado inicial q_0 , estados finais $F = \{q_1\}$ e cuja função de transição é dada por:

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_0

- (a) Esboce o diagrama de estados deste autômato.
 - (b) Descreva a computação deste autômato que tem início na configuração $(q_0, aabba)$. Esta palavra é aceita pelo autômato?
 - (c) Descreva a computação deste autômato que tem início na configuração $(q_0, aabbab)$. Esta palavra é aceita pelo autômato?
 - (d) Descreva em português a linguagem aceita pelo autômato definido acima?
4. Seja Σ um alfabeto com n símbolos. Quantos autômatos finitos determinísticos existem com alfabeto Σ e $m > 0$ estados?
- Sugestão:** Não esqueça de considerar todas as possibilidades para o conjunto de estados finais.
5. Invente autômatos finitos determinísticos que aceitem as seguintes linguagens sobre o alfabeto $\{0, 1\}$:
- (a) o conjunto das palavras que acabam em 00;
 - (b) o conjunto das palavras com três 0s consecutivos;
 - (c) o conjunto das palavras em que cada 0 está entre dois 1s;
 - (d) o conjunto das palavras cujos quatro símbolos finais são 1101;
 - (e) o conjunto dos palíndromos de comprimento igual a 6.
6. Dê exemplo de uma linguagem que é aceita por um autômato finito determinístico com *mais de um estado final*, mas que *não* é aceita por nenhum autômato finito determinístico com *apenas um estado final*. Justifique cuidadosamente sua resposta.

Expressões regulares

É hora de abordarmos o primeiro dos problemas propostos ao final do capítulo anterior; isto é, como determinar a linguagem aceita por um autômato finito? De quebra, descobriremos uma maneira de caracterizar estas linguagens.

1. Introdução

Desejamos obter um algoritmo que, dado um autômato finito $\mathcal{M}$, determine a linguagem $L(\mathcal{M})$ que ele aceita. O algoritmo é recursivo, e para poder descrevê-lo começamos por generalizar a noção de linguagem aceita por um autômato.

Seja $\mathcal{M}$ um autômato finito definido pelos ingredientes $(\Sigma, Q, q_1, F, \delta)$. Para cada $q \in Q$ definimos L_q como sendo a linguagem

$$L_q = \{w \in \Sigma^* : (q, w) \vdash^* (f, \epsilon) \text{ onde } f \in F\}.$$

Em outras palavras, L_q é formada pelas palavras que levam o autômato do estado q a algum estado final. Quando os estados do autômato forem numerados como $q_1, \dots, q_n$, escrevermos L_i em vez de L_{q_i} para simplificar a notação. Portanto, se q_1 é o estado inicial de $\mathcal{M}$, então

$$L_{q_1} = L_1 = L(\mathcal{M}).$$

Sejam p e q estados de $\mathcal{M}$, e digamos que $\delta(p, \sigma) = q$. Esta transição estabelece uma relação entre as linguagens L_p e L_q . De fato, temos que $(p, \sigma) \vdash (q, \epsilon)$ ao passo que, se $w \in L_q$, então $(q, w) \vdash^* (f, \epsilon)$. Combinado estas duas computações obtemos

$$(p, \sigma w) \vdash (q, w) \vdash^* (f, \epsilon).$$

Portanto, $\{\sigma\}L_q \subseteq L_p$. Mais uma vez, com a finalidade de não sobrecarregar a notação, eliminaremos as chaves, escrevendo simplesmente $\sigma L_q \subseteq L_p$.

A não ser que o alfabeto Σ tenha apenas um símbolo, não há a menor chance de que $\sigma L_q \subseteq L_p$ seja uma igualdade. Isto porque teremos uma inclusão como esta para cada $\sigma \in \Sigma$. Portanto, se $\Sigma = \{\sigma_1, \dots, \sigma_n\}$ e se $\delta(p, \sigma_i) = q_i$, então

$$\bigcup_{i=1}^n \sigma_i L_{q_i} \subseteq L_p.$$

Desta vez, porém, a inclusão é, de fato, uma igualdade, desde que p não seja um estado final do autômato! Para ver isto suponhamos que $u \in L_p$. Podemos isolar o primeiro símbolo de u , escrevendo $u = \sigma_i w$, para algum $\sigma_i \in \Sigma$. Mas, pela definição de L_p ,

$$(p, u) = (p, \sigma_i w) \vdash^* (f, \epsilon),$$

para algum $f \in F$. Por outro lado, como $\delta(p, \sigma_i) = q_i$, temos que $(p, \sigma_i) \vdash (q_i, \epsilon)$. Combinando estas duas computações, temos que

$$(p, u) = (p, \sigma_i w) \vdash (q_i, w) \vdash^* (f, \epsilon),$$

de onde segue que $w \in L_{q_i}$.

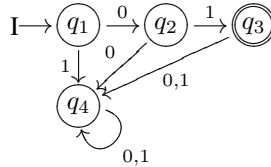
Precisamos ainda analisar o que acontece quando p é um estado final do autômato. Considerando em detalhe o argumento do parágrafo acima, vemos que assumimos implicitamente que $u \neq \epsilon$, já que estamos explicitando o seu primeiro símbolo. Entretanto, se p for um estado final, teremos, além disso, que $\epsilon \in L_p$.

Resumindo, provamos que, se $\Sigma = \{\sigma_1, \dots, \sigma_n\}$, e se $\delta(p, \sigma_i) = q_i$, então

$$L_p = \begin{cases} \bigcup_{i=1}^n \sigma_i L_{q_i} & \text{se } p \notin F \\ \bigcup_{i=1}^n \sigma_i L_{q_i} \cup \{\epsilon\} & \text{se } p \in F \end{cases}$$

O algoritmo que desejamos segue diretamente desta equação, como mostra o seguinte exemplo.

Considere o autômato finito determinístico $\mathcal{M}$ com alfabeto $\{0, 1\}$ e cujo grafo é



Deste grafo extraímos as seguintes equações

$$L_1 = 0L_2 \cup 1L_4$$

$$L_2 = 1L_3 \cup 0L_4$$

$$L_3 = 0L_4 \cup 1L_4 \cup \{\epsilon\} \quad (\text{já que } q_3 \text{ é estado final})$$

$$L_4 = 0L_4 \cup 1L_4.$$

Observe que q_4 é um estado morto, de modo que nada escapa de q_4 . Em particular, nenhuma palavra leva o autômato de q_4 a um estado final. Portanto, $L_4 = \emptyset$. Isto simplifica drasticamente as equações anteriores, que passam a ser

$$\begin{aligned} L_1 &= 0L_2 \\ L_2 &= 1L_3 \\ L_3 &= \{\epsilon\} \\ L_4 &= \emptyset. \end{aligned}$$

Podemos agora resolver este sistema de equações por mera substituição. Assim, substituindo a terceira equação na segunda, obtemos

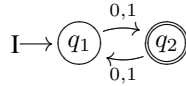
$$L_2 = 1L_3 = 1\{\epsilon\} = \{1\}.$$

Finalmente, substituindo esta última equação na primeira, obtemos

$$L_1 = 0L_2 = 1\{0\} = \{10\}.$$

Como $L_1 = L(\mathcal{M})$, obtivemos uma descrição da linguagem aceita por $\mathcal{M}$.

Como seria de esperar, as coisas nem sempre são tão diretas. Afinal, o autômato deste exemplo tinha um comportamento muito simples. Vejamos o que acontece em um exemplo menos elementar. Por exemplo, seja $\mathcal{N}$ o autômato finito determinístico de alfabeto $\{0, 1\}$ e grafo



As equações correspondentes a L_1 e L_2 são:

$$\begin{aligned} L_1 &= 0L_2 \cup 1L_2 \\ L_2 &= 0L_1 \cup 1L_1 \cup \{\epsilon\} \quad (\text{já que } q_2 \text{ é estado final}) \end{aligned}$$

Continuando a denotar $\{0\}$ e $\{1\}$ por 0 e 1, podemos reescrever estas equações como

$$\begin{aligned} L_1 &= (0 \cup 1)L_2 \\ L_2 &= (0 \cup 1)L_1 \cup \{\epsilon\}. \end{aligned}$$

À primeira vista, tudo o que temos que fazer para resolver este sistema é substituir a segunda equação na primeira. Contudo, ao fazer isto obtemos

$$L_1 = (0 \cup 1)((0 \cup 1)L_1 \cup \{\epsilon\}),$$

ou seja,

$$(1.1) \quad L_1 = (0 \cup 1)^2 L_1 \cup (0 \cup 1),$$

de modo que L_1 fica escrito em termos do próprio L_1 . Parece claro que nenhuma substituição pode nos tirar desta encrenca. O que fazer então?

2. Lema de Arden

Na verdade, ao aplicar o método de substituição para resolver sistemas de equações e achar a linguagem aceita por um autômato vamos nos deparar muitas vezes com equações em que uma linguagem é escrita em termos dela própria. Equações que serão, freqüentemente, ainda mais complicadas que (1.1). Convém, portanto, abordar deste já o problema em um grau de generalidade suficiente para dar cabo de todas as equações que apareçam como resultado do algoritmo de substituição.

Para isso, suponhamos que Σ é um alfabeto e que A e B são linguagens em Σ . Digamos, que X seja uma outra linguagem em Σ que satisfaça

$$X = AX \cup B.$$

O problema que queremos resolver consiste em usar esta equação para determinar X .

Uma coisa que podemos fazer é substituir $X = AX \cup B$ de volta nela própria. Isso dá

$$(2.1) \quad X = A(AX \cup B) \cup B = A^2X \cup (A \cup \epsilon)B,$$

e não parece adiantar de nada porque afinal de contas continuamos com um X do lado direito da equação. Mas não vamos nos deixar abater por tão pouco: façamos a substituição mais uma vez. Desta vez substituíremos $X = AX \cup B$ em (2.1), o que nos dá

$$X = A^2(AX \cup B) \cup (AB \cup B) = A^3X \cup (A^2 \cup A \cup \epsilon)B.$$

Repetindo o mesmo procedimento k vezes, chegamos à equação

$$X = A^{k+1}X \cup (A^k \cup A^{k-1} \cup \dots \cup A^2 \cup A \cup \epsilon)B.$$

O problema é que o X continua presente. Mas, e se repetíssemos o processo infinitas vezes? Neste caso, “perderíamos de vista o termo que contém X que seria empurrado para o infinito”, e sobraria apenas

$$(2.2) \quad X = (\epsilon \cup A \cup A^2 \cup \dots)B.$$

Para poder fazer isto de maneira formal precisamos introduzir uma nova operação com linguagens, a *estrela de Kleene*. Em geral, se A é uma linguagem no alfabeto Σ , então A^* é definida como a reunião de todas as potências de A ; isto é,

$$A^* = \{\epsilon\} \cup A \cup A^2 \cup A^3 \cup \dots.$$

Por exemplo, se $\Sigma = \{0, 1\}$ e $A = \{01\}$, então

$$A^* = \{(01)^j : j \geq 0\} = \{\epsilon, 01, 0101, 010101, \dots\}.$$

Por outro lado, se $A = \Sigma$, então A^* é o conjunto de todas as palavras no alfabeto Σ —que aliás já vínhamos denotando por Σ^* .

A equação (2.2) sugere que, continuando o processo de substituição indefinidamente, deveríamos obter $X = A^*B$. Este é um conjunto perfeitamente bem definido, resta-nos verificar se realmente é uma solução da equação

$X = AX \cup B$. Para isso substituíremos X por A^*B do lado direito da equação:

$$AX \cup B = A(A^*B) \cup B = AA^*B \cup B.$$

Como $A^* = \{\epsilon\} \cup A \cup A^2 \cup A^3 \cup \dots$, obtemos

$$AA^*B \cup B = A(\{\epsilon\} \cup A \cup A^2 \cup A^3 \cup \dots)B \cup B,$$

que dá

$$AA^*B \cup B = (A \cup A^2 \cup A^3 \cup A^4 \cup \dots)B \cup B.$$

Pondo B em evidência em todo o lado direito, obtemos

$$AA^*B \cup B = (\epsilon \cup A \cup A^2 \cup A^3 \cup A^4 \cup \dots)B = A^*B.$$

Concluimos que $A(A^*B) \cup B = A^*B$, de modo que A^*B é, de fato, uma solução da equação $X = AX \cup B$.

Infelizmente, isto ainda não é suficiente para completar os cálculos do algoritmo de substituição. O problema é que obtivemos uma solução da equação desejada, mas ainda não sabemos se esta solução corresponde ao maior conjunto $X \subseteq \Sigma^*$ que satisfaz $X = AX \cup B$. Se não for este o caso, quando usarmos A^*B como solução estaremos perdendo algumas palavras do conjunto aceito pelo autômato, o que não queremos que aconteça.

Suponhamos, então, que X é o maior subconjunto de Σ^* que satisfaz $X = AX \cup B$. Como já sabemos que A^*B satisfaz esta equação, podemos escrever $X = A^*B \cup C$, onde C é um conjunto (disjunto de A^*B) que contém as possíveis palavras excedentes. Substituindo $X = A^*B \cup C$ em $X = AX \cup B$, temos

$$(2.3) \quad A^*B \cup C = A(A^*B \cup C) \cup B = A^*B \cup AC,$$

já que, como vimos, $A(A^*B) \cup B = A^*B$. Intersectando ambos os membros de (2.3) com C , e lembrando que, por hipótese, $C \cap A^*B = \emptyset$, chegamos a

$$C = AC \cap C = (A \cap \epsilon)C.$$

Temos, então, duas possibilidades. A primeira é que A não contenha ϵ . Neste caso $A \cap \epsilon = \emptyset$, de modo que $C = \emptyset$; ou seja, A^*B é o maior conjunto solução da equação $X = AX \cup B$. A outra possibilidade é que A contenha ϵ , e neste caso estamos encrocados. Por sorte, esta segunda possibilidade *nunca* ocorre na solução das equações que advêm do algoritmo de substituição! Você pode confirmar isto lendo a demonstração detalhada do algoritmo de substituição na seção 5. Vamos resumir tudo o que fizemos em um lema, provado originalmente por D. N. Arden em 1960.

LEMA DE ARDEN. *Sejam A e B linguagens em um alfabeto Σ . Se $\epsilon \notin A$ então o maior subconjunto de Σ^* que satisfaz $X = AX \cup B$ é $X = A^*B$.*

Vamos aplicar o que aprendemos para resolver a equação (1.1), que resultou da aplicação do método de substituição ao segundo exemplo da seção anterior. A equação é

$$L_1 = (0 \cup 1)^2 L_1 \cup (0 \cup 1).$$

Aplicando o Lema de Arden com $X = L_1$, $A = (0 \cup 1)^2$ e $B = (0 \cup 1)$, teremos que

$$L_1 = X = A^*B = ((0 \cup 1)^2)^*(0 \cup 1).$$

Concluimos, assim, que a linguagem aceita pelo autômato $\mathcal{N}$ é

$$L(\mathcal{N}) = ((0 \cup 1)^2)^*(0 \cup 1).$$

3. O algoritmo de substituição

Antes de fazer outro exemplo, vamos descrever em mais detalhes o algoritmo de substituição que usamos para determinar a linguagem aceita pelos autômatos da seção 1. Este algoritmo foi inventado por J. A. Brzozowski em 1964.

Algoritmo de substituição

Entrada: ingredientes $(\Sigma, Q, q_1, F, \delta)$ de um autômato finito determinístico $\mathcal{M}$.

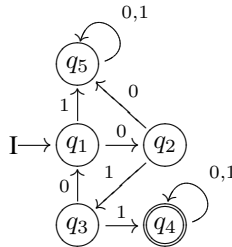
Saída: uma descrição da linguagem aceita por $\mathcal{M}$.

Primeira etapa: Seja $Q = \{q_1, \dots, q_n\}$. Escreva as equações para as linguagens L_j , para cada $1 \leq j \leq n$.

Segunda etapa: Começando por L_n e acabando em L_1 , substitua L_{j+1} na equação para L_j , aplicando o Lema de Arden sempre que uma linguagem for expressa em termos dela própria.

Terceira etapa: A linguagem aceita por $\mathcal{M}$ corresponde à expressão obtida para L_1 .

Uma descrição detalhada deste algoritmo, e uma demonstração de que faz o que é pedido, pode ser encontrada na seção 5. Por enquanto, nos contentaremos com a descrição acima, que vamos aplicar ao autômato no alfabeto $\{0, 1\}$ cujo grafo é



As equações correspondentes a autômato são

$$L_1 = 0L_2 \cup 1L_5$$

$$L_2 = 1L_3 \cup 0L_5$$

$$L_3 = 0L_1 \cup 1L_4$$

$$L_4 = 0L_4 \cup 1L_4 \cup \epsilon$$

$$L_5 = 0L_5 \cup 1L_5.$$

Uma olhada no grafo mostra que q_5 é um estado morto, de modo que $L_5 = \emptyset$. Com isto as equações se simplificam:

$$\begin{aligned} L_1 &= 0L_2 \\ L_2 &= 1L_3 \\ L_3 &= 0L_1 \cup 1L_4 \\ L_4 &= 0L_4 \cup 1L_4 \cup \epsilon \\ L_5 &= \emptyset. \end{aligned}$$

A equação para L_4 nos dá

$$L_4 = (0 \cup 1)L_4 \cup \epsilon,$$

de modo que precisamos aplicar o Lema de Arden. fazendo isto obtemos

$$L_4 = (0 \cup 1)^* \epsilon = (0 \cup 1)^*.$$

Substituindo em L_3 ,

$$L_3 = 0L_1 \cup 1(0 \cup 1)^*.$$

De modo que, da segunda equação, segue que

$$L_2 = 1(0L_1 \cup 1(0 \cup 1)^*) = 10L_1 \cup 11(0 \cup 1)^*.$$

Com isso, resulta da primeira equação que

$$L_1 = 010L_1 \cup 011(0 \cup 1)^*.$$

Usando o Lema de Arden mais uma vez,

$$L_1 = 010^*(011(0 \cup 1)^*),$$

que é a linguagem aceita pelo autômato.

Antes de encerrar a seção precisamos fazer algumas considerações sobre a aplicação do Lema de Arden.

Em primeiro lugar, o que aconteceria se não tivéssemos notado que q_5 é um estado morto? Neste caso teríamos de confrontar a equação $L_5 = 0L_5 \cup 1L_5$, ou seja $L_5 = (0 \cup 1)L_5$. Como L_5 aparece dos dois lados da equação, será necessário aplicar o Lema de Arden. Note que, neste caso $X = L_5$, $A = (0 \cup 1)$ e $B = \emptyset$, de modo que

$$L_5 = X = (0 \cup 1)^* \emptyset = \emptyset,$$

que é o resultado esperado.

O segundo comentário diz respeito à aplicação do Lema de Arden à equação

$$L_4 = 0L_4 \cup 1L_4 \cup \epsilon.$$

Na aplicação que fizemos anteriormente, tomamos $A = (0 \cup 1)$ e $B = \epsilon$. Mas o que aconteceria se escolhêssemos $A = 0$ e $B = 1L_4 \cup \epsilon$? Neste caso,

$$L_4 = 0^*(1L_4 \cup \epsilon) = 0^*1L_4 \cup 0^*,$$

e continuamos com L_4 dos dois lados da equação. Mas, ao invés de nos deixar intimidar, aplicaremos o Lema de Arden a esta última equação, o que nos dá

$$(3.1) \quad L_4 = (0^*1)^*0^*.$$

Note que se isto estiver correto (e está!) então devemos ter que os conjuntos $(0^*1)^*0^*$ e $(0 \cup 1)^*$ são iguais—quer dizer, têm os mesmos elementos. Portanto, deve ser possível mostrar que toda palavra no alfabeto $\{0, 1\}$ pertence ao conjunto $(0^*1)^*0^*$. Fica por sua conta se convencer disto. Finalmente, se adotarmos esta última maneira de expressar L_4 , a descrição da linguagem aceita pelo autômato que resulta do algoritmo de substituição é

$$010^*(011(0^*1)^*0^*).$$

Em particular, o algoritmo de substituição pode retornar linguagens diferentes, todas corretas, dependendo da maneira como for aplicado.

4. Expressões regulares

Utilizando o algoritmo de substituição sempre obtemos uma descrição bastante precisa da linguagem aceita por um autômato finito determinístico. Além disso, por causa da maneira como o algoritmo opera, a linguagem é quase sempre expressa como resultado da aplicação das operações de união, concatenação e estrela, aos conjuntos unitários formados pelos símbolos do alfabeto do autômato. As únicas exceções ocorrem quando a linguagem é vazia ou é ϵ . Formalizaremos isto em uma definição, como segue.

Seja Σ um alfabeto e seja

$$\tilde{\Sigma} = \Sigma \cup \{\cup, \cdot, *, (,), \emptyset, \epsilon\}.$$

Consideraremos o conjunto $\tilde{\Sigma}$ como um outro alfabeto, uma extensão de Σ . Além disso, $\cup, \cdot, *, (,), \emptyset, \epsilon$ serão considerados apenas como símbolos (isto é, seu significado será ignorado) quando estiverem posando de elementos de $\tilde{\Sigma}$.

Uma *expressão regular* é uma palavra no alfabeto $\tilde{\Sigma}$, construída recursivamente pela aplicação sucessiva das seguintes regras:

- (1) se $\sigma \in \Sigma$ então σ é uma expressão regular;
- (2) $\emptyset$ e ϵ são expressões regulares;
- (3) se r_1 e r_2 são expressões regulares, então $(r_1 \cup r_2)$ e $(r_1 \cdot r_2)$ também são;
- (4) se r é uma expressão regular, então r^* também é.

Não há nada de misterioso sobre (3) e (4), elas apenas refletem a maneira correta de se usar os símbolos $\cup, \cdot$ e $*$, quando são interpretados como operadores de conjuntos. Portanto, se identificarmos $0, 1, \epsilon$ e $\emptyset$ com os conjuntos $\{0\}, \{1\}, \{\epsilon\}$ e $\emptyset$, uma expressão regular $r \in \tilde{\Sigma}^*$ corresponderá a um subconjunto $L(r)$ em Σ , a linguagem *denotada* pela expressão regular r .

Suponhamos, por exemplo, que $\Sigma = \{0, 1\}$. Neste caso

$$\tilde{\Sigma} = \{0, 1, \cup, \cdot, *, (,), \emptyset, \epsilon\}.$$

Como 0 e 1 são expressões regulares por (1), então 0^* também é uma expressão regular por (4). Mas isto implica, por (3), que $(0^* \cdot 1)$ é regular. Usando (4) novamente, obtemos $(0^* \cdot 1)^*$, e por (3) concluímos que $((0^* \cdot 1)^* \cdot 0^*)$ é uma expressão regular. Esta é a expressão regular que denota uma das maneiras de representar a linguagem L_4 do final da seção 3. Entretanto, para obter uma expressão regular corretamente construída, precisamos acrescentar parêntesis à representação de L_4 dada pela equação (3.1). O papel dos parêntesis é apenas o de eliminar qualquer ambigüidade na interpretação das expressões.

A razão para introduzir expressões regulares como palavras em um alfabeto, em vez de pensá-las simplesmente como a descrição de uma linguagem, é que expressões regulares distintas podem denotar o mesmo conjunto. Este é o caso, por exemplo, das expressões $(0 \cup 1)^*$ e $((0^* \cdot 1)^* \cdot 0^*)$, como vimos ao final da seção anterior.

É claro que qualquer conjunto que possa ser representado a partir dos conjuntos unitários ϵ e $\sigma \in \Sigma$ e das operações de união, concatenação e estrela pode ser denotado por uma expressão regular. Em particular, se cuidarmos de pôr os parêntesis no lugar certo, o algoritmo de substituição aplicado a um autômato finito $\mathcal{M}$ sempre retorna uma expressão regular que denota a linguagem $L(\mathcal{M})$.

Resta-nos praticar um pouco a construção de uma expressão regular que denote um conjunto dado, a partir da descrição deste conjunto. Suponhamos que $\Sigma = \{a, b, c\}$. Para obter *todas as palavras* em um certo subconjunto de Σ devemos usar a estrela de Kleene. Assim,

Linguagem formada por todas as palavras	Expressão regular
(vazias ou não) que só contêm a	a^*
nos símbolos a, b e c	$((a \cup b) \cup c)^*$
que não contêm a	$(b \cup c)^*$
em a e cujo comprimento é par	$(a \cdot a)^*$

A última expressão merece um comentário. As palavras de comprimento par têm a forma a^{2k} , para algum $k \geq 0$ inteiro. Mas,

$$a^{2k} = (aa)^k \in \{aa\}^k.$$

Portanto, o conjunto das palavras de comprimento par será

$$\bigcup_{k \geq 0} \{aa\}^k = aa^*.$$

Já as palavras no símbolo a cujo comprimento é ímpar são da forma $a^{2k+1} = a \cdot (aa)^k$. Assim, a linguagem formada por essas palavras é denotada por $a \cdot (a \cdot a)^*$.

Outro exemplo interessante consiste na linguagem formada pelas palavras que contêm exatamente um a . Isto significa que os outros símbolos da palavra têm que ser bs ou cs . Como estes símbolos tanto podem aparecer antes como

depois do a , uma tal palavra será da forma uav , onde u e v são palavras que contêm apenas b e c . Isto nos remete à expressão regular $((b \cup c)^* \cdot a \cdot (b \cup c)^*)$.

Como as expressões regulares já estão ficando bastante complicadas, vamos suprimir os parêntesis e o ponto que denota concatenação, quando não forem absolutamente necessários à interpretação correta da expressão. Evidentemente, ao fazer isto não obtemos uma expressão regular no sentido da definição formal. Fica como exercício acrescentar os símbolos necessários para que cada uma das expressões dadas abaixo se torne uma expressão regular correta.

Duas variações, dignas de nota, do último exemplo acima são a linguagem formada por todas as palavras que contêm exatamente dois as , que corresponde a

$$(b \cup c)^* a (b \cup c)^* a (b \cup c)^*$$

e a linguagem formada por todas as palavras que contêm um número par de as , que é denotada por

$$((b \cup c)^* a (b \cup c)^* a (b \cup c)^*)^*$$

Um exemplo um pouco mais difícil é a linguagem formada pelas palavras que contêm um número ímpar de as . Precisamos adicionar um a extra à expressão acima. O problema é que este a pode aparecer em qualquer lugar da palavra, de modo que não podemos simplesmente concatená-lo no início ou no fim da expressão acima. A saída é tomar

$$((b \cup c)^* a (b \cup c)^* a (b \cup c)^*)^* a ((b \cup c)^* a (b \cup c)^* a (b \cup c)^*)^*.$$

Encerramos com três exemplos de natureza mais prática.

Um dos primeiros passos do processo de compilação de uma linguagem de programação é conhecido como *análise léxica*. Nesta etapa o compilador identifica, por exemplo, quais foram os números inteiros e as variáveis utilizadas no programa. É claro que esta é uma etapa necessária para que seja possível interpretar corretamente o programa. Na prática, isto pode ser feito com um autômato finito. Assim, para identificar as variáveis, construímos um autômato finito que aceita a linguagem que descreve as variáveis de uma linguagem. Isto é feito em duas etapas. Primeiro obtemos uma expressão regular que denote as variáveis da linguagem de programação. Em seguida, construímos um autômato finito que aceite esta linguagem.

Contudo, pôr esta estratégia em prática depende de sermos capazes de resolver algorítmicamente o seguinte problema.

PROBLEMA 3.1. *Dada uma expressão regular r , construir um autômato finito que aceite a linguagem denotada por r .*

Abordaremos este problema detalhadamente a partir do próximo capítulo. Entretanto, já estamos em condições de obter expressões regulares para as linguagens que descrevem inteiros e variáveis de um programa, como veremos a seguir.

Para começar determinaremos uma expressão regular no alfabeto $\{0, 1, \dots, 9\}$ que denote os inteiros positivos no sistema decimal. À primeira vista pode parecer que a expressão seja simplesmente

$$(0 \cup 1 \cup \dots \cup 9)^*.$$

O problema é que isto inclui palavras como 00000, que não correspondem a um número formado de maneira correta. A solução é não permitir que as palavras comecem com o símbolo 0, tomando

$$(1 \cup \dots \cup 9)(0 \cup 1 \cup \dots \cup 9)^*.$$

Já a expressão que denota os inteiros não negativos é

$$0 \cup (1 \cup \dots \cup 9)(0 \cup 1 \cup \dots \cup 9)^*.$$

Finalmente, suponhamos que uma certa linguagem de programação tem por variáveis todas as palavras nos símbolos $0, 1, \dots, 9, A, B, C, \dots, Z$ que não começam por um número inteiro. A expressão regular que denota as variáveis nesta linguagem de programação é

$$(A \cup B \cup \dots \cup Z)(0 \cup 1 \cup \dots \cup 9)^*.$$

5. Análise formal do algoritmo de substituição

Nesta seção damos uma descrição detalhada do algoritmo de substituição; isto é, uma descrição cuidadosa o suficiente para servir, tanto para programar o algoritmo, quanto para provar que funciona como esperado. Na verdade, encerramos a seção justamente com uma demonstração de que o algoritmo está correto. Contudo, se você não pretende programar o algoritmo, nem sente necessidade de uma demonstração formal, talvez seja melhor pular esta seção e passar ao próximo capítulo.

Começamos com algumas definições um tanto técnicas. Para $t = 0, \dots, m$ seja Σ_t o alfabeto definido por

$$\Sigma_t = \begin{cases} \Sigma & \text{se } t = 0 \\ \Sigma \cup \{\alpha_1, \dots, \alpha_t\} & \text{se } t \geq 1. \end{cases}$$

Seja agora Θ_t o conjunto formado pelas expressões regulares em Σ_t da forma

$$\bigcup_{i \leq t} \eta_i \cdot \alpha_i,$$

onde $\eta_i \neq \epsilon$ é uma expressão regular em $\Sigma_0 = \Sigma$. Note que Θ_0 é o conjunto das expressões regulares em Σ .

Dado $\theta \in \Theta_t$, seja $L(\theta)$ a linguagem obtida de acordo com as seguintes regras:

- (1) $L(\epsilon) = \{\epsilon\}$;
- (2) $L(\emptyset) = \emptyset$;
- (3) $L(\alpha_i) = L_i$;

com L_i como definido na seção 1, e se α e β são expressões regulares em Σ_t então

- (4) $L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta)$;
- (5) $L(\alpha \cup \beta) = L(\alpha) \cup L(\beta)$;
- (6) $L(\alpha^*) = L(\alpha)^*$.

Estamos, agora, prontos para dar uma descrição minuciosa do funcionamento do algoritmo.

Algoritmo de Substituição

Entrada: um autômato finito determinístico $\mathcal{A}$ cujos ingredientes são $(\Sigma, Q, q_1, F, \delta)$, onde $Q = \{q_1, \dots, q_n\}$.

Saída: uma expressão regular que denota a linguagem aceita por $\mathcal{A}$.

Etapa 1: Para cada estado q_i escreva uma expressão regular em Σ_m da forma

$$E_i = \bigcup \{(\sigma_j \cdot \alpha_{j(i)}) : \delta(q_i, \sigma_j) = q_{j(i)}\}$$

se q_m não é estado final, ou

$$E_i = \bigcup \{(\sigma_j \cdot \alpha_{j(i)}) : \delta(q_i, \sigma_j) = q_{j(i)}\} \cup \{\epsilon\}$$

se q_m é estado final; e inicialize $k = m$.

Etapa 2: Se E_k já está escrito como uma expressão regular em Θ_{k-1} , vá para (3), senão E_k é da forma

$$E_k = \eta \cdot \alpha_k \cup B$$

onde $B \in \Theta_{k-1}$. Neste caso escreva $E_k = \eta^* \cdot B$ e vá para a Etapa 3. Observe que pode acontecer que $B = \emptyset$; se isto ocorrer então $E_k = \emptyset$.

Etapa 3: Subtraia 1 de k . Se $k = 0$, então $L(\mathcal{A})$ é denotada pela expressão regular E_1 no alfabeto Σ_0 e podemos parar; senão, substitua α_k por E_k na expressão regular E_i para $i \neq k$ e volte à Etapa 2.

Resta-nos apenas dar uma demonstração de que este algoritmo funciona. Faremos isso usando indução finita.

DEMONSTRAÇÃO. Digamos que, para um certo inteiro $0 \leq k \leq m$ estamos para executar o $(m - k)$ -ésimo laço deste algoritmo. Queremos mostrar que, ao final deste laço

- (1) $E_i \in \Theta_{k-1}$ e
- (2) $L(E_i) = L_i$,

para $i = 1, \dots, m$. Para fazer isto, podemos supor que, chegados a este laço, temos $E_i \in \Theta_k$ e $L(E_i) = L_i$ para $i = 1, \dots, m$. Observe que estas duas últimas afirmações são claramente verdadeiras quando $k = m$.

Começamos considerando a execução da Etapa 2 no $(m - k)$ -ésimo laço. Se já temos que $E_k \in \Theta_{k-1}$ então nada há a fazer nesta etapa. Por outro lado, se $E_k \notin \Theta_{k-1}$, então como $E_k \in \Theta_k$ podemos escrever

$$(5.1) \quad E_k = \eta_k \cdot \alpha_k \cup B,$$

onde $B \in \Theta_{k-1}$. Seja $F = \eta_k^* \cdot B$. Temos de (5.1) e (2) que

$$A_k = L(E_k) = L(\eta_k) \cdot A_k \cup L(B).$$

Logo, pelo lema de Arden,

$$A_k = L(\eta_k)^* \cdot L(B),$$

mas isto é igual a $L(F)$. Como o algoritmo manda fazer $E_k = F$, obtemos, ao final desta etapa, que $E_k \in \Theta_{k-1}$ e $L(E_k) = A_k$.

Finalmente, na etapa 3, basta substituir α_k por F na expressão regular de E_j sempre que $j \neq k$. Note que segue imediatamente disto que $E_j \in \Theta_{k-1}$ e que $L(E_j) = A_j$.

6. Exercícios

- Seja $\Sigma = \{0, 1\}$. Se $L_1 = \{0\}$ e $L_2 = \{1\}^*$, determine:
 - $L_1 \cup L_2$ e $L_1 \cap L_2$;
 - L_1^* e L_2^* ;
 - $(L_1 \cup L_2)^*$;
 - $L_1^* \cap L_2$;
 - $\overline{L_1}$;
 - $\overline{L_2} \cap L_1^*$.
- Se L é uma linguagem em um alfabeto Σ então $L^R = \{w^R : w \in L\}$. Se $L = \{0\} \cdot \{1\}^*$ calcule L^R .
- Seja L uma linguagem no alfabeto Σ . O que podemos concluir a respeito de L se $L^+ = L^* \setminus \{\epsilon\}$?
- Sejam Σ_1 e Σ_2 dois alfabetos e seja $\phi : \Sigma_1 \rightarrow \Sigma_2^*$ uma aplicação. Se L é uma linguagem no alfabeto Σ_1 , mostre que $\phi(L^*) = \phi(L)^*$.
- Para cada um dos autômatos finitos determinísticos, no alfabeto $\{0, 1\}$, dados abaixo:
 - esboce o diagrama de estados;
 - encontre os sorvedouros e os estados mortos;
 - determine a expressão regular da linguagem aceita pelo autômato usando o algoritmo de substituição.

- (a) Os estado são $\{q_1, \dots, q_4\}$, o estado inicial é q_1 , o conjunto de estados finais é $\{q_2\}$ e a função de transição é dada por:

δ	0	1
q_1	q_2	q_4
q_2	q_3	q_1
q_3	q_4	q_4
q_4	q_4	q_4

- (b) Os estado são $\{q_1, \dots, q_5\}$, o estado inicial é q_1 , o conjunto de estados finais é $\{q_3, q_4\}$ e a função de transição é dada por:

δ	0	1
q_1	q_2	q_4
q_2	q_2	q_3
q_3	q_5	q_5
q_4	q_5	q_5
q_5	q_5	q_5

- (c) Os estado são $\{q_1, \dots, q_4\}$, o estado inicial é q_1 , o conjunto de estados finais é $\{q_1\}$ e a função de transição é dada por:

δ	0	1
q_1	q_2	q_4
q_2	q_3	q_1
q_3	q_4	q_2
q_4	q_4	q_4

- (d) Os estado são $\{q_1, q_2, q_3\}$, o estado inicial é q_1 , o conjunto de estados finais é $\{q_1\}$ e a função de transição é dada por:

δ	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_1	q_2

- (e) Os estado são $\{q_1, \dots, q_6\}$, o estado inicial é q_1 , o conjunto de estados finais é $\{q_4\}$ e a função de transição é dada por:

δ	0	1
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_3
q_4	q_4	q_4
q_5	q_6	q_2
q_6	q_6	q_4

6. Descreva em português o conjunto denotado por cada uma das expressões regulares abaixo:

- (a) 1^*0 ;
- (b) $1^*0(0)^*$
- (c) $111 \cup 001$;
- (d) $(1 \cup 00)^*$;
- (e) $(0(0)^*1)^*$;
- (f) $(0 \cup 1)(0 \cup 1)^*00$;

7. Expresse cada uma das seguintes linguagens no alfabeto $\{0, 1\}$ usando uma expressão regular:

- (a) o conjunto das palavras de um ou mais zeros seguidos de um 1;

- (b) o conjunto das palavras de dois ou mais símbolos seguidos por três ou mais zeros;
 - (c) o conjunto das palavras que contêm uma sequência de 1s, de modo que o número de 1s na sequência é congruente a 2 módulo 3, seguido de um número par de zeros.
8. Se r e s são expressões regulares, vamos escrever $r \equiv s$ se e somente se $L(r) = L(s)$. Supondo que r , s e t são expressões regulares, mostre que:
- (a) $(r \cup r) \equiv r$;
 - (b) $((r \cdot s) \cup (r \cdot t)) \equiv (r \cdot (s \cup t))$;
 - (c) $((s \cdot r) \cup (t \cdot r)) \equiv ((s \cup t) \cdot r)$;
 - (d) $(r^* \cdot r^*) \equiv r^*$;
 - (e) $(r \cdot r^*) \equiv (r^* \cdot r)$;
 - (f) $r^{**} \equiv r^*$;
 - (g) $(\epsilon \cup (r \cdot r^*)) \equiv r^*$;
 - (h) $((r \cdot s)^* \cdot r) \equiv (r \cdot (s \cdot r)^*)$;
 - (i) $(r \cup s)^* \equiv (r^* \cdot s^*)^* \equiv (r^* \cup s^*)^*$.

9. Usando as identidades do exercício 4 prove que

$$((abb)^*(ba)^*(b \cup aa)) \equiv (abb)^*((\epsilon \cup (b(ab)^*a))b \cup (ba)^*(aa)).$$

Observe que alguns parênteses e o símbolo “.” foram omitidos para facilitar a leitura.